

Relatório de Atividades: Integração de Sistemas Distribuídos (Java Spring Boot e PHP)

Disciplina: Sistemas Distribuídos

Instituição: UFC - Campus Quixadá

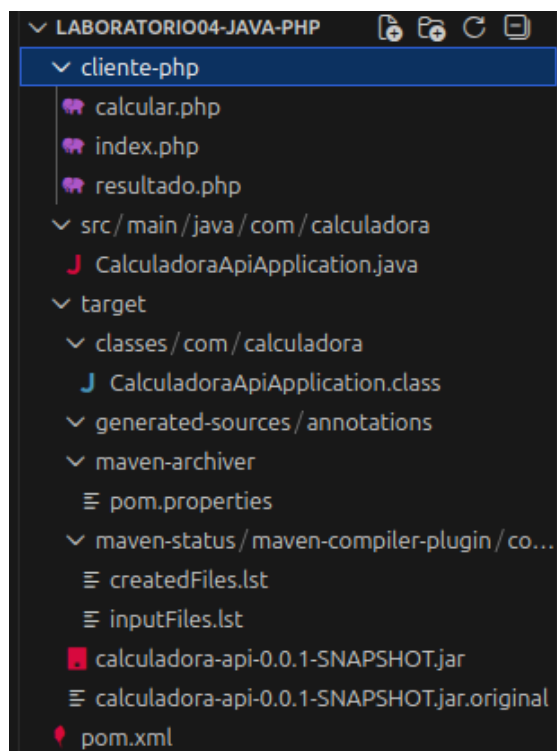
Autor(a): Gabriely Correia Dealem

1. Introdução e Objetivo

Este relatório descreve o desenvolvimento de um sistema distribuído composto por duas camadas distintas: um **WebService REST** desenvolvido em Java (Spring Boot) e um **Cliente Web** desenvolvido em PHP. O objetivo principal foi implementar a comunicação entre tecnologias diferentes via protocolo HTTP, permitindo a execução de operações aritméticas de forma remota.

2. Arquitetura do Sistema

A solução segue o modelo **Cliente-Servidor**:

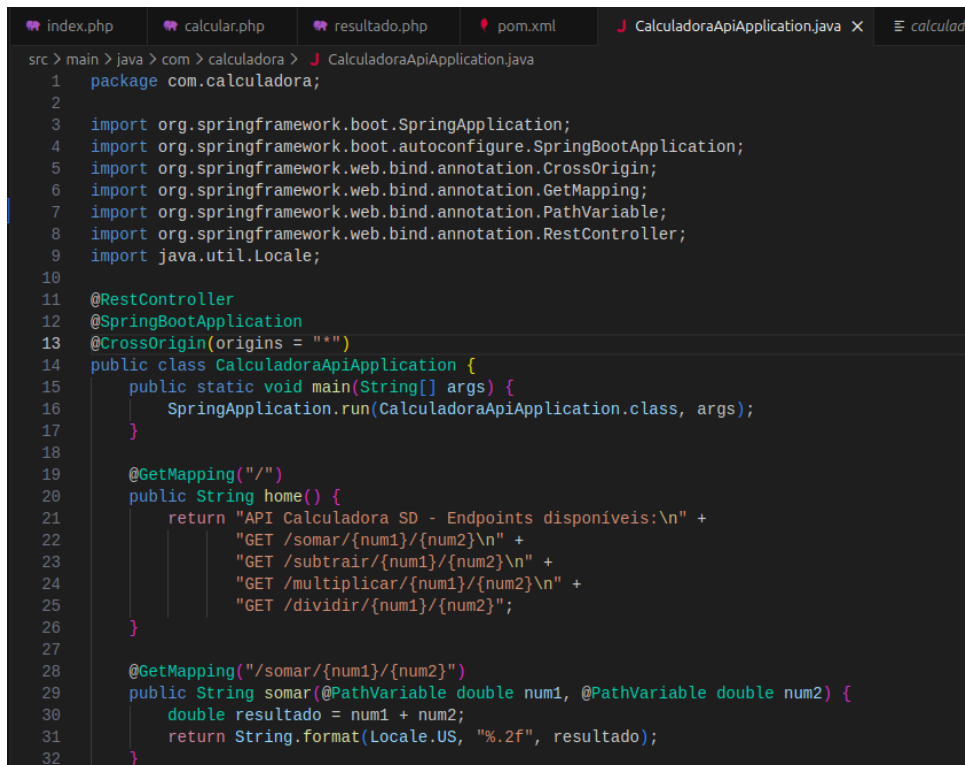


- **Servidor (Back-end):** Uma API REST em Spring Boot que processa os cálculos.
- **Cliente (Front-end):** Uma interface PHP que coleta os dados do usuário, consome a API via cURL e exibe o resultado.

3. Implementação do WebService (Java Spring Boot)

O serviço foi construído em uma única classe controladora (**CalculadoraApiApplication**) utilizando as seguintes anotações:

- **@RestController**: Define a classe como um componente que lida com requisições HTTP e retorna dados diretamente no corpo da resposta.
- **@CrossOrigin(origins = "**")**: Permite que o cliente PHP (rodando em uma porta diferente) acesse os recursos da API sem bloqueios de política de mesma origem (CORS).
- **Endpoints**: Foram mapeados quatro endpoints principais (**/somar**, **/subtrair**, **/multiplicar**, **/dividir**) utilizando **@GetMapping** e **@PathVariable** para captura de parâmetros dinâmicos.



```
src > main > java > com > calculadora > J CalculadoraApiApplication.java
1  package com.calculadora;
2
3  import org.springframework.boot.SpringApplication;
4  import org.springframework.boot.autoconfigure.SpringBootApplication;
5  import org.springframework.web.bind.annotation.CrossOrigin;
6  import org.springframework.web.bind.annotation.GetMapping;
7  import org.springframework.web.bind.annotation.PathVariable;
8  import org.springframework.web.bind.annotation.RestController;
9  import java.util.Locale;
10
11  @RestController
12  @SpringBootApplication
13  @CrossOrigin(origins = "**")
14  public class CalculadoraApiApplication {
15      public static void main(String[] args) {
16          SpringApplication.run(CalculadoraApiApplication.class, args);
17      }
18
19      @GetMapping("/")
20      public String home() {
21          return "API Calculadora SD - Endpoints disponíveis:\n" +
22              "GET /somar/{num1}/{num2}\n" +
23              "GET /subtrair/{num1}/{num2}\n" +
24              "GET /multiplicar/{num1}/{num2}\n" +
25              "GET /dividir/{num1}/{num2}";
26      }
27
28      @GetMapping("/somar/{num1}/{num2}")
29      public String somar(@PathVariable double num1, @PathVariable double num2) {
30          double resultado = num1 + num2;
31          return String.format(Locale.US, "%.2f", resultado);
32      }
33  }
```

```

    }

    @GetMapping("/subtrair/{num1}/{num2}")
    public String subtrair(@PathVariable double num1, @PathVariable double num2) {
        double resultado = num1 - num2;
        return String.format(Locale.US, "%.2f", resultado);
    }

    @GetMapping("/multiplicar/{num1}/{num2}")
    public String multiplicar(@PathVariable double num1, @PathVariable double num2) {
        double resultado = num1 * num2;
        return String.format(Locale.US, "%.2f", resultado);
    }

    @GetMapping("/dividir/{num1}/{num2}")
    public String dividir(@PathVariable double num1, @PathVariable double num2) {
        if (num2 == 0) {
            return "ERRO: Divisao por zero nao permitida";
        }
        double resultado = num1 / num2;
        return String.format(Locale.US, "%.2f", resultado);
    }
}

```

4. Implementação do Cliente (PHP)

O cliente foi dividido em três arquivos para melhor organização:

1. **index.php**: Formulário HTML para entrada de números e seleção da operação.

```

index.php x  calculador.php  resultado.php  pom.xml  CalculadoraApiApplication.java  calculador
cliente-php > index.php
1  <!DOCTYPE html>
2  <html lang="pt-br">
3  <head>
4      <meta charset="UTF-8">
5      <title>Cliente PHP - Calculadora Spring Boot</title>
6      <style>
7          body { font-family: Arial, sans-serif; margin: 40px; }
8          .container { max-width: 400px; margin: auto; }
9          input, select, button { margin: 10px 0; padding: 10px; width: 100%; }
10         button { background: #28a745; color: white; border: none; cursor: pointer; }
11         button:hover { background: #218838; }
12     </style>
13 </head>
14 <body>
15     <div class="container">
16         <h2>Calculadora - Consumo API Spring Boot</h2>
17         <form action="calculador.php" method="GET">
18             <input type="number" step="any" name="num1" placeholder="Numero 1" required>
19             <input type="number" step="any" name="num2" placeholder="Numero 2" required>
20             <select name="operacao">
21                 <option value="somar">Somar</option>
22                 <option value="subtrair">Subtrair</option>
23                 <option value="multiplicar">Multiplicar</option>
24                 <option value="dividir">Dividir</option>
25             </select>
26             <button type="submit">Calcular</button>
27         </form>
28     </div>
29 </body>
30 </html>

```

2. **calculador.php**: Script lógico que utiliza a biblioteca **cURL** para realizar a requisição **GET** ao servidor Java, captura o retorno e redireciona o fluxo.

```
index.php  calcular.php X  resultado.php  pom.xml  J CalculadoraApiApplication.java

cliente-php > calcular.php
1  <?php
2  $num1 = $_GET['num1'] ?? 0;
3  $num2 = $_GET['num2'] ?? 0;
4  $operacao = $_GET['operacao'] ?? 'somar';
5
6  $url = "http://localhost:8080/$operacao/$num1/$num2";
7
8  $ch = curl_init();
9  curl_setopt($ch, CURLOPT_URL, $url);
10 curl_setopt($ch, CURLOPT_RETURNTRANSFER, true);
11 curl_setopt($ch, CURLOPT_TIMEOUT, 5);
12
13 $resposta = curl_exec($ch);
14
15 if (curl_errno($ch)) {
16     $erro = "Erro na conexao com a API: " . curl_error($ch);
17     curl_close($ch);
18     header("Location: index.php?erro=" . urlencode($erro));
19     exit();
20 }
21
22 curl_close($ch);
23 header("Location: resultado.php?resultado=" . urlencode($resposta));
24 ?>
```

3. **resultado.php**: Página de apresentação que exibe o valor final processado ou mensagens de erro.

```
index.php  calcular.php  resultado.php X  pom.xml  J CalculadoraApiApplication.java  calculadora-

cliente-php > resultado.php
1  <!DOCTYPE html>
2  <html lang="pt-br">
3  <head>
4      <meta charset="UTF-8">
5      <title>Resultado</title>
6      <style>
7          body { font-family: Arial, sans-serif; margin: 40px; }
8          .container { max-width: 400px; margin: auto; text-align: center; }
9          .resultado { font-size: 24px; margin: 20px 0; padding: 20px; background: #f8f9fa; }
10         a { color: #28a745; text-decoration: none; }
11     </style>
12 </head>
13 <body>
14     <div class="container">
15         <h2>Resultado da Operacao</h2>
16         <div class="resultado">
17             <?php
18             if (isset($_GET['resultado'])) {
19                 echo htmlspecialchars($_GET['resultado']);
20             } else {
21                 echo "Nenhum resultado encontrado.";
22             }
23             ?>
24         </div>
25         <a href="index.php">Nova Operacao</a>
26     </div>
27 </body>
28 </html>
```

5. Procedimentos de Execução

Para a correta orquestração do ambiente distribuído, foram utilizados os seguintes comandos em terminais distintos:

5.1. Inicialização do Back-end (Java)

Terminal 1: No diretório raiz do projeto Java (onde se localiza o arquivo `pom.xml`), executou-se o comando para subir o servidor na porta padrão `8080`:

- `mvn spring-boot:run`

5.2. Inicialização do Front-end (PHP)

Terminal 2: No diretório do cliente, iniciou-se o servidor de desenvolvimento do PHP na porta `8000`:

- `php -S localhost:8000 -t cliente-php/`

6. Desafios e Resolução de Problemas

Durante o desenvolvimento, a principal dificuldade encontrada foi a **conectividade**. O erro de conexão foi resolvido ao garantir que ambos os servidores estivessem ativos simultaneamente em portas diferentes (`8000` e `8080`). Além disso, a configuração do Maven no arquivo `pom.xml` foi ajustada para reconhecer corretamente o plugin do Spring Boot, permitindo a execução do comando de *build*.

7. Validação dos Resultados

A validação foi realizada submetendo valores através da interface PHP. Por exemplo:

- **Entrada:** `num1=10, num2=5, operacao=somar`
- **Requisição Interna:** `GET http://localhost:8080/somar/10/5`
- **Resposta da API:** `15.00`
- **Exibição no Navegador:** "Resultado da Operação: 15.00"

8. Conclusão

A atividade demonstrou com sucesso a interoperabilidade entre Java e PHP. A utilização de uma arquitetura baseada em serviços REST permitiu que a lógica de negócio ficasse centralizada no servidor Spring Boot, enquanto a interface de usuário pôde ser desenvolvida de forma independente em PHP, características fundamentais de sistemas distribuídos modernos.